Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Yapısal (structural) kalıplara giriş
  - Yaratımsal kalıplar bir önceki derste tamamlanmıştı. Bu derste Dörtlü Çete'nin (GoF) yapısal kalıplarından Adapter, Bridge ve Composite işlendi.
- Adapter kalıbı: amaç ve kullanım
  - Bir istekçi (client) sınıf, bir hizmet sınıfının metodunu belli bir adla çağırmak istiyor; aynı işi yapan metot hizmet sınıfında başka bir adla (başka bir imzayla) var. İki sınıfın kaynak kodu da değiştirilemiyor.
  - Çözüm: araya istekçinin beklediği metot imzasına sahip bir sınıf (adaptör) konur. Bu metodun gövdesi, asıl işi yapan sınıfın ilgili metodunu çağırır; çoğu zaman tek satırlık bir yönlendirme (delegasyon) kodudur.
  - Benzetme: farklı ülkelerin priz standartları farklıdır (ör. İngiltere'de üç bacaklı fiş). Seyahat adaptörü, iki bacaklı fişi üç delikli prize takılabilir hâle getirir.
  - Kullanım koşulu: elimizdeki sınıfın istekçi tarafından kullanılmasının önündeki tek engel, metot adlarının (imzalarının) farklı olmasıdır.
- Adapter örneği: çizim programı ve XXCircle (ekrandan, 00:10:00 ve 00:11:00)

```
Shape {setLocation(), getLocation(), display(), fill(), setColor(), undisplay()}
   △── Point, Line, Square   {display(), fill(), undisplay()}
   △── Circle  ◆──── XXCircle {displayIt(), fillIt(), undisplayIt(),
                               setLocation(), getLocation(), setItsColor()}
```

```
class Circle extends Shape {
    ...
    private XXCircle myXXCircle;
    ...
    public Circle() {
        myXXCircle = new XXCircle();
    }
    public void display() {
        myXXCircle.displayIt();
    }
    ...
}
```

  - Nokta, çizgi ve kare kodlanmıştır; daire çizmek (yumuşak kenar, anti-aliasing vb.) zor bir iştir. Başka bir projede yazılmış `XXCircle` sınıfı vardır ama metot adları farklıdır (`display` yerine `displayIt`, `fill` yerine `fillIt`). Kaynak kodu kaybolmuş, yalnız `.class` dosyaları kalmış olabilir.
  - `Circle` adaptör sınıfıdır: `Shape`'ten kalıtılır, her metodunun içinde `XXCircle`'ın karşılık gelen metodunu çağırır.
  - `Circle` ile `XXCircle` arasındaki ilişki birebir sahipliktir. Slaytta dolu elmas çizilmiştir; hoca düz bir sahiplik okunun yeterli olduğunu, elmasın isteğe bağlı olduğunu söyledi.
  - Ders sonrası not: Slayttaki kodda `void public display()` yazıyor (ekrandan, 00:11:00); Java'da doğru sıra `public void display()` biçimindedir.

- Bridge kalıbı: amaç, GoF örneği ve yapı (ekrandan, 00:20:00 ve 00:22:30)
  - Amaç: bir nesnenin soyutlamasını (arayüzünü) gerçeklemesinden ayırmak; böylece ikisi bağımsız olarak çeşitlenebilir. Windows programlamasında "Handle" olarak da bilinir.
  - GoF örneği: farklı masaüstü/pencere yönetim ortamlarını (X Window, Presentation Manager; hoca GNOME ve KDE'yi de örnekledi) destekleyen bir GUI çerçevesi. Tek kalıtım hiyerarşisinde yeni bir pencere türü (ör. sistem çubuğuna küçülen `IconWindow`) eklemek, her ortam için ayrı bir alt sınıf (`XIconWindow`, `PMIconWindow` ...) gerektirir.
  - Çözüm: pencere soyutlaması (`Window` → `IconWindow`, `TransientWindow`) ile pencere gerçeklemesi (`WindowImp` → `XWindowImp`, `PMWindowImp`) iki ayrı kalıtım hiyerarşisine konur. Aralarındaki birebir sahiplik ilişkisi (`imp`) köprüdür. `Window.DrawRect()` dört kez `imp->DevDrawLine()` çağırır.

```
Abstraction {Operation()}  ◇── imp ──>  Implementor {OperationImp()}
   △── RefinedAbstraction                 △── ConcreteImplementorA {OperationImp()}
                                         △── ConcreteImplementorB {OperationImp()}
Operation():  imp->OperationImp();
```

  - Bileşenler: Abstraction uygulamanın temel işlemlerini oluşturan arayüzü belirler; RefinedAbstraction arayüzü özelleştirir; Implementor uygulamanın iç yapısına yönelik arayüzü belirler; ConcreteImplementor iç yapıyı gerçekleyen sınıf hiyerarşisidir.
  - Implementor bir arayüz (interface) olarak tanımlanabilir; Abstraction tanımlanamaz, çünkü gerçekleme nesnesini tutar ve alt sınıfların ortak işlerini gerçeklemenin metotları üzerinden kodlar.
  - Kullanım: soyutlama ile gerçekleme arasındaki bağın kalıcı olması istenmiyorsa (çalışma anında değiştirilebilmeli), ya da soyutlamaların ve gerçeklemelerin ayrı ayrı özelleşme ihtiyacı varsa. Kalıtım çalışma anında değiştirilemeyen bir ilişkidir.

- Bridge örneği: iki çizim kütüphanesiyle dikdörtgen ve daire (ekrandan, 00:30:00–00:45:00)
  - Problem: bir vektör çizim programında iki dosya standardı var. DP1'de dikdörtgen koordinatları `x1, y1, x2, y2`, DP2'de `x1, x2, y1, y2` sırasında verilir. DP1 ve DP2, hazır statik çizim metotlarına sahip iki kod kütüphanesidir (`DP1.draw_a_line()`, `DP2.drawline()`).
  - Önerilen çözüm (1): soyut `Rectangle` (`+draw()`, `#drawLine()`), alt sınıfları `V1Rectangle` (DP1 kullanır) ve `V2Rectangle` (DP2 kullanır).

```
package bridge.solution1;
public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;
    public void draw() {
        drawLine(_x1, _y1, _x2, _y1);
        drawLine(_x2, _y1, _x2, _y2);
        drawLine(_x2, _y2, _x1, _y2);
        drawLine(_x1, _y2, _x1, _y1);
    }
    abstract protected void drawLine(double x1, double y1,
                                     double x2, double y2);
}
public class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
                            double x2, double y2) {
        DP1.draw_a_line(x1, y1, x2, y2);
    }
}
```

  - `drawLine` nesnenin iç çalışmasıyla ilgili olduğu için public olmamalıdır; private olursa alt sınıflara geçmez. Bu yüzden protected yapılır (UML'de #): alt sınıflara public, diğer sınıflara private gibi davranır.

- Daire de eklenip ortak `Shape` üst sınıfı konunca (her şekil tek bir döngüde `draw()` ile çizilebilsin diye) tasarım `V1Rectangle`, `V2Rectangle`, `V1Circle`, `V2Circle` olur.
- Zayıf yön (ekrandan, 00:36:30): 2 aile × 2 şekil = 4 sınıf; yeni bir çizim kütüphanesi (DP3) eklenince 3 × 2 = 6; üstüne üçgen eklenince 3 × 3 = 9. Her yeni çizim metodu veya şekil türüyle sınıf sayısı fırlar.
- Bridge ile çözüm (ekrandan, 00:40:00):

```
Shape Abstraction                          Drawing Implementation
Shape {+draw() abstract,      ◇── _dp ──> Drawing {+drawLine() abstract,
       #drawLine(), #drawCircle()}                    +drawCircle() abstract}
   △── Rectangle {+draw()}                    △── V1Drawing {+drawLine(), +drawCircle()} ·
   △── Circle    {+draw()}                    △── V2Drawing {+drawLine(), +drawCircle()} ·
```

```java
package bridge.solution2;
public abstract class Shape {
    public abstract void draw();
    private Drawing _dp;
    public Shape(Drawing dp) { _dp = dp; }
    protected void drawLine(double x1, double y1,
                            double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x, double y,
                              double r) { _dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle(Drawing dp, double x, double y, double r) {
        super(dp);
        _x = x; _y = y; _r = r;
    }
    public void draw() { drawCircle(_x, _y, _r); }
}
public abstract class Drawing {
    public abstract void drawLine(double x1, double y1, double x2, double y2);
    public abstract void drawCircle(double x, double y, double r);
}
public class V1Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x, y, r);
    }
    public void drawLine(double x1, double y1, double x2, double y2) {
        DP1.draw_a_line(x1, y1, x2, y2);
    }
}
```

(kod: ekrandan, 00:42:00 ve 00:45:00)

- `Shape.draw()` soyuttur, ama `drawLine` ve `drawCircle` somut ve protected'dır: gövdeleri yalnız ilişkili gerçekleme nesnesine (`_dp`) yönlendirme yapar. `Drawing`'in metotları ise soyuttur; nasıl çizileceği aileye (V1/V2) göre değişir.
- Kazanç: üçgen eklemek için yalnız `Shape`'in bir alt sınıfı (`Triangle`, üç kez `drawLine`), DP3 eklemek için yalnız `Drawing`'in bir alt sınıfı (`V3Drawing`) eklenir. Her yeni ihtiyaç için tek sınıf.

- Bridge üzerine öğrenci tartışması (01:05–01:33)
  - Bir öğrenci `Shape`'te `drawLine`/`drawCircle` bulunmasının ona fazla sorumluluk yüklediğini, `_dp` protected yapılarak alt sınıfların doğrudan kullanabileceğini önerdi.
  - Hoca Violet UML Editor'de alternatif tasarımı çizdi: soyut `DP` (`+drawLine()`), DP1 ve DP2'nin kaynak kodu olmadığı için `DP1_Adaper` ve `DP2_Adaper` adaptörleriyle (Adapter kalıbı); `Shape(d: DP)` kurucusu ve `Rectangle.draw()` içinde dört kez `d.drawLine()`. İki çözümde de sınıf sayısı 8 çıktı.
  - Hangi tasarımın daha iyi olduğu ancak kod yazıldıktan sonra bağlaşım (coupling) ve uyum (cohesion) metrikleriyle objektif olarak söylenebilir; iskelet kod buna yetmez.

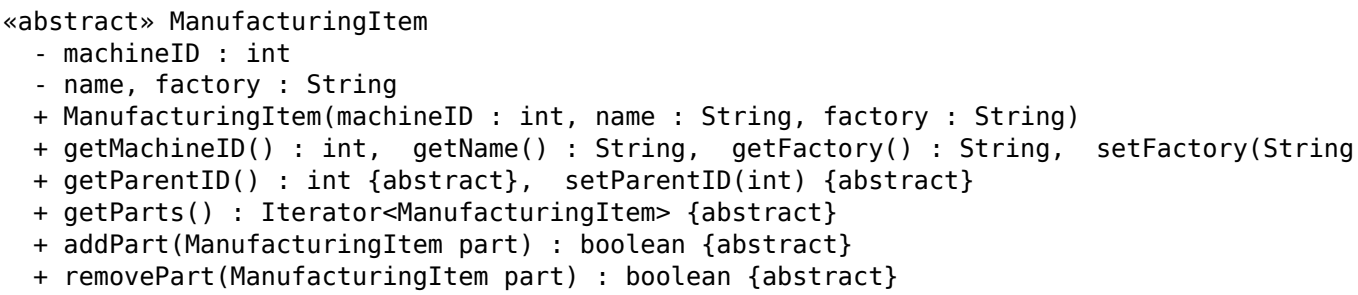
  - İkinci soru: varlık (entity) sınıflarına `draw` gibi davranışlar yüklemek doğru mu? Hocanın karşı örneği: ayrı bir `Drawer` sınıfı (`drawRectangle(Rectangle)`, `drawCircle(Circle)`, `drawLine(...)`) `Rectangle`'ın getter'larını sürekli çağırır. Bu, "feature envy" ve "long parameter list" kod kusurlarına yol açar (ekrandan, 01:30:00); `Rectangle` yalnız getter'ı olan bir veri nesnesine dönüşür.
  - Hocanın ilkesi: bir sorumluluk, onu yerine getirmek için gereken verinin en çoğuna sahip olan sınıfa atanır; böylece nesneler arasında en az veri aktarılır.

- Composite kalıbı: amaç, örnek ve yapı (ekrandan, 01:36:00 ve 01:40:00)
  - Amaç: hiyerarşik parça-bütün ilişkilerini desteklemek; tek nesnelere ve nesne gruplarına ortak bir biçimde erişmek.
  - Örnek: Word/PowerPoint'te çizgileri gruplayıp alt resim yapmak, grupları yeniden gruplamak. Grup, iç düzeni bozulmadan taşınabilir.
  - Örnek çözüm: soyut `Graphic` (`Draw()`, `Add(Graphic)`, `Remove(Graphic)`, `GetChild(int)`); yapraklar `Line`, `Rectangle`, `Text`; bileşik `Picture` `graphics` listesini tutar. `Picture.Draw()`: `forall g in graphics: g.Draw()`; `Add(g)`: g'yi listeye ekle. Bire-çok parça-bütün ilişkisinde elmas kullanımı zorunludur.
  - Kalıp yapısı: `Component` (`Operation()`, `Add`, `Remove`, `GetChild`), `Leaf`, `Composite` (`children`). Component ortak kullanım arayüzü ile parçalara erişim/düzenleme metotlarını tanımlar; Leaf hiyerarşinin en altındaki, çocuğu olamayan düğümdür; Composite ara düğümdür (farklı tür ara düğümler varsa alt sınıfları olarak modellenir). Client parçaları ve toplulukları aynı biçimde kullanır; slayta göre Client bir Visitor olabilir.
  - Terimler: ağaçta en alttaki, çocuğu olamayan düğüm yaprak; en tepedeki kök; diğerleri ara düğümdür.

- Composite gerçekleme ayrıntıları
  - Çocuk ebeveynini bilmeli mi? Gerekirse üst düğümden, kökten ya da ikisinden haberdar edilir; haberdarlık arttıkça tutarlılık denetimleri ağırlaşır.
  - Düğümü kim, nasıl siler? Köke "şunu sil" denebilir ya da düğüme "kendini sil" denebilir; ikincisi için düğümün üst düğümü ya da kökü bilmesi gerekir. Ara düğüm silinince çocukları da mı silinir, yoksa üst ebeveyne mi bağlanır? Bu, gerçek dünya problemine göre gereksinimlerde belirlenir.
  - Hangi ağaç veri yapısı (ikili ağaç, kırmızı-siyah ağaç vb.) kullanılacağı da ayrı bir karardır.
  - Çocuk listesi Composite yerine Component'te tutulabilir; ama o zaman Component'i arayüz yapma esnekliği kaybolur.

- Composite'in zayıflığı
  - `Add`, `Remove`, `GetChild` Composite için anlamlıdır, Leaf için anlamsızdır; ama kalıtımda üst sınıfın metotları reddedilemez. Leaf'te boş gövdeli metotlar ya da `return null` yazmak gerekir. Kalıtımın özü alt sınıfın üst sınıfın özelleşmiş hâli olmasıdır; burada durum tersidir.
  - Programcının vermesi gereken karar sayısı diğer kalıplara göre çok fazladır.

- Composite uygulama örneği: makine üretim kataloğu (ekrandan, 02:10:00–02:30:00)
  - Problem: birden fazla fabrikada makineler ve makine parçaları üretiliyor; katalogdan farklı raporlar istenir (ör. belli bir fabrikada üretilenler). Makineler alt makine ve parçalar içerebilir. Örnek Design Patterns Java Workbook kitabından esinlenmiştir.



`MachinePart ──▷ ManufacturingItem  (MachinePart(machineID, parentID, name, factory))`
`Report 1 ··> * ManufacturingItem;  MainApp ··> Report`

  - Hoca `MachinePart`'a bir `parentID` üye alanı eklenmesi gerektiğini, fabrikanın da aslında `String` değil ayrı bir sınıf olması gerektiğini söyledi (örneği sade tutmak için böyle bırakılmış).
  - Bir parça birden fazla makinede yer alacaksa, yalnız `parentID`'si farklı kopyalar oluşturulur. Tek nesne şartı (nesne kimliği önemliyse) aradaki ilişki için ek bir sınıf gerektirirdi.
  - `Machine` (ekrandan, 02:20:00): `getParentID() return 0;`, `setParentID` boş gövdeli; `addPart` önce `part.setParentID(getMachineID())` çağırır, parça listede varsa `false` döner, yoksa `parts.add(part)` sonucunu döner; `removePart` `parts.remove(part)`, `getParts` `parts.iterator()` döner.
  - `MachinePart`: `parentID` için getter/setter; `getParts()` `null`, `addPart` ve `removePart` `false` döner (kalıbın zayıflığının somut görünümü).
  - `Report.findItems(items[], factory)` statik metottur; her öğenin fabrikasını `compareToIgnoreCase` ile karşılaştırır, eşleşeni numarayla yazar, ardından öğenin `getParts()` iteratörüyle alt parçalarını da tarar. `MachinePart.getParts()` `null` döndürdüğü için `subItems == null` denetimi gerekir.
  - Çalıştırma (ekrandan, 02:30:00): Bursa için çıktı `99 Krank şaftı KŞ-7`, `259 Flanş FTR-5`, `63 Distribütör DST-12`, "3 parça bulundu". Jeneratör İstanbul'da üretilir ama parçası Flanş Bursa'dadır.

- Kod kusuru: tekrarlanan kod (duplicate code)
  - `Report`'ta ana öğeyi ve alt parçayı yazdıran satırlar neredeyse aynıdır (12–17 ve 22–26. satırlar; kopyala-yapıştır). Bir değişiklik iki yerde yapılmak zorunda kalır.
  - `ReportV2`'de yazdırma tek bir `showItem` metodunda toplandı; bunun bedeli, sıra sayacının yerel değişken yerine üye alan olmasıdır. Çıktı aynıdır.

Hocanın Özellikle Vurguladığı Kısımlar

- Adapter yalnız ad/imza uyumsuzluğunu çözer
  - Adaptör sınıfı işi baştan yazmaz; işlevleri hazır sınıfa iletir (delege eder). Her uyumsuz sınıf için ayrı bir adaptör gerekir (ör. `XXPentagon` için ayrı bir adaptör).
- Bridge: soyutlama ile gerçeklemeyi iki hiyerarşiye ayırmak
  - Bir kısım kalıtım ilişkisi sahiplik (parça-bütün) ilişkisine çevrilir; kalıtım tamamen kaldırılmaz. Sonuç: her yeni şekil ya da her yeni çizim ailesi için yalnız bir sınıf eklenir.
  - "Neyin değiştiğini bul ve onu sarmala" ilkesi: farklı biçimde evrimleşen şeyler ayrı kalıtım ağaçlarına konur.
  - Abstraction arayüz olamaz, Implementor olabilir.
- Protected erişim belirleyicisi
  - İç çalışmayla ilgili ama alt sınıflara geçmesi gereken metotlar protected yapılır; UML'de # ile gösterilir.
- Tasarım seçimi ölçülerek yapılır
  - Adapter mı Bridge mi sorusuna hoca önceden kesin cevap vermedi: doğru karar ancak kodlanmış tasarımların bağlaşım ve uyum metrikleriyle karşılaştırılmasıyla ya da benzer bir problemdeki deneyimle verilebilir.
  - Sorumluluk, gerekli verinin en çoğuna sahip sınıfa atanır; yalnız getter/setter'ı olan veri nesneleri ve başka nesnenin verisine sürekli erişen metotlar (feature envy) kusurdur.
- Composite kalıtımın özüne aykırıdır
  - Leaf, anlamsız `add/remove/getChild` metotlarını boş gövde ya da `null` ile taşımak zorundadır. Buna rağmen hiyerarşik parça-bütün ilişkilerini modellemekte kullanışlı olduğu için tercih edilir.
  - Ebeveyn bilgisi, silme sorumluluğu, ara düğüm silinince çocukların durumu ve veri yapısı seçimi önceden, gereksinimlere göre kararlaştırılmalıdır.
- Kullanılan veri yapısının ve platformun ayrıntılarını bilmek gerekir
  - Örnekte `getParts()`'ın `null` döndürmesi yüzünden null denetimi gerekti.

Kısa Tekrar Notları

- Adapter: istekçinin beklediği imzayı sunan sınıf, asıl işi hazır sınıfın farklı adlı metoduna iletir. Örnek: `Circle.display()` → `myXXCircle.displayIt()`.
- Bridge: soyutlama (`Shape`, `Window`) ve gerçekleme (`Drawing`, `WindowImp`) iki ayrı hiyerarşide; aralarında `imp`/`_dp` sahiplik köprüsü.
- Bridge'siz tasarımda sınıf sayısı aile × şekil kadar büyür (2×2=4, 3×2=6, 3×3=9); Bridge ile yeni şekil ya da yeni aile tek sınıf demektir.
- Bridge kullanım koşulları: soyutlama-gerçekleme bağı çalışma anında değişebilmeli ya da ikisi ayrı ayrı özelleşmeli.
- Composite: Component (ortak arayüz + add/remove/getChild), Leaf (çocuksuz), Composite (çocuk listesi, işlemi çocuklarına iletir).
- Composite'in zayıflığı: Leaf'te anlamsız metotlar (boş gövde, `return null`, `return false`).
- Composite kararları: ebeveyn bilgisi, silme, ara düğüm silinince çocuklar, veri yapısı, çocuk listesinin yeri.
- Makine kataloğu: `ManufacturingItem` (soyut), `Machine` (composite, `ArrayList`), `MachinePart` (leaf, `parentID`), `Report.findItems`.
- Kod kusurları: duplicate code, feature envy, long parameter list (çözüm: preserve whole object), yalnız veri tutan sınıf.

Detaylı Açıklamalar

- Adapter kalıbı, iki hazır yazılım parçasını değiştirmeden birbirine bağlamanın en basit yoludur. İstekçi `Target` arayüzünün `request()` metodunu çağırmak ister; aynı işi yapan `Adaptee` sınıfında ise metot `specificRequest()` adındadır. `Target`'ı gerçekleyen (ya da ondan kalıtılan) bir adaptör sınıfı yazılır; `request()` metodunun gövdesi yalnız `adaptee.specificRequest()` çağrısıdır. Çizim programı örneğinde `Shape`'in alt sınıfı olarak `Circle` yazılır ve `XXCircle` nesnesi `Circle`'ın private üye alanı olarak tutulur. `Circle`'ın her metodu `XXCircle`'ın karşılığını çağırır. Bu yaklaşımın sınırı da açıktır: her yeni uyumsuz sınıf (elips, beşgen …) için yeni bir adaptör sınıfı yazmak gerekir.
- Bridge kalıbında asıl sorun, iki bağımsız değişim ekseninin tek kalıtım ağacında birleştirilmesidir. Şekil türleri bir eksen, çizim kütüphaneleri başka bir eksendir; tek ağaçta her birleşim ayrı bir sınıf olur ve sınıf sayısı çarpımsal büyür. Bridge iki ekseni iki ayrı hiyerarşiye ayırır: `Shape` ve alt sınıfları "ne çizilecek" bilgisini, `Drawing` ve alt sınıfları "nasıl çizilecek" bilgisini taşır. `Shape`, kurucusunda aldığı `Drawing` nesnesini `_dp` alanında tutar; `Rectangle.draw()` dört kez `drawLine`, `Circle.draw()` bir kez `drawCircle` çağırır. Bu çağrılar `Shape`'teki protected metotlar üzerinden `_dp`'ye, oradan da DP1 ya da DP2'nin statik metotlarına gider. Gerçekleme nesnesi çalışma anında verildiği için, aynı `Rectangle` sınıfı farklı `Drawing` nesneleriyle farklı kütüphanelerle çizilebilir. Hoca, hiyerarşiyi kalıtımla değil sahiplik ilişkisiyle bağlamanın "kalıtım yerine parça-bütün ilişkisini tercih et" ilkesinin bir uygulaması olduğunu vurguladı.
- Öğrenci tartışmasında iki alternatif yan yana kondu. Bridge çözümünde `Shape`, gerçeklemeye yönlendirme yapan `drawLine`/`drawCircle` metotlarını taşır; bunlarda iş mantığı yoktur, yalnız `_dp` üzerinden aktarma yapılır. Adapter'lı alternatifte soyut bir `DP` sınıfı ve DP1, DP2 için adaptörler kurulur; `Shape(d: DP)` kurucusu bir adaptör alır ve şekiller `d.drawLine()` çağırır. DP1 ve DP2'nin kaynak kodu elde olmadığı için bunlar doğrudan `DP`'den kalıtılamaz; adaptör bu yüzden gereklidir. Hoca iki çözümde de sınıf sayısının aynı (8) olduğunu, hangisinin daha esnek olduğuna ancak kodlanmış hâlde bağlaşım ve uyum ölçülerek karar verilebileceğini söyledi.
- Varlık sınıflarına davranış yükleme sorusunda hoca, çizimi ayrı bir `Drawer` sınıfına taşımanın sonuçlarını gösterdi. `drawRectangle(Rectangle r)` metodu `drawLine(r.getX1(), r.getX2(), ...)` gibi sürekli başka nesnenin verisine erişir (feature envy), çizim metotlarının parametre sayısı uzar (long parameter list) ve `Rectangle` yalnız getter'ları olan bir veri nesnesine dönüşür. Long parameter list için normalde "preserve whole object" yeniden düzenlemesi (refactoring) önerilir; burada `drawRectangle` zaten tüm nesneyi aldığı için bu çözüm bir şey kazandırmaz. Hoca, sorumluluğun gereken verinin çoğunu tutan sınıfa (burada koordinatları tutan şekle) verilmesini daha doğru buldu; farklı çizim aileleri olduğu için de bunun Bridge ya da Adapter ile esnekleştirilmesi gerektiğini ekledi.

- Composite kalıbı ağaç biçimindeki parça-bütün ilişkilerini tek bir ortak üst tür üzerinden modeller. `Composite`, `Component` türünden bir çocuk listesi tutar; bu sayede çocukları hem yaprak hem başka bileşik olabilir. `Composite.Operation()` çağrısı listedeki her çocuğa iletilir; yapraklar işi kendileri yapar, bileşikler yeniden çocuklarına iletir. Böylece istemci tek bir nesneyle bir nesne ağacını aynı biçimde kullanır. İşlemler çizim olabileceği gibi bir makinenin toplam maliyetini ya da üretim süresini hesaplamak gibi raporlamalar da olabilir.
- Makine kataloğu örneğinde `Machine` composite, `MachinePart` leaf rolündedir. `Machine` için anlamsız olan `setParentID` boş gövdelidir ve `getParentID` 0 döner; `MachinePart` için anlamsız olan `getParts` `null`, `addPart`/`removePart` `false` döner. Hoca bunları kalıbın kalıtımın özüne aykırı yanının somut örneği olarak gösterdi. Rapor sınıfında makinenin kendisi aranan fabrikada üretilmese bile alt parçalarından biri orada üretilmiş olabileceği için alt parçalar da taranır. Hoca son olarak `Report`'taki kopyala-yapıştır kodu "duplicate code" kod kusuru olarak işaretledi ve `ReportV2`'de yazdırmayı `showItem` metodunda topladı.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.